Appeal-Desk — Judge Test Script

A self-contained walkthrough a judge can run against the live app or the repo. ~10 minutes end-to-end. Every step has a concrete pass criterion that doesn't depend on subjective taste.

Why this document exists

Judging hackathon submissions is hard when "it works on my machine" stops at the README. This script is a deterministic harness: it tells you exactly what to type, what you should see, and what counts as a pass. If any step is ambiguous, that's a bug in this script — not in your reading.

The script is grounded in the actual repo. Every command, file path, and expected number is reproducible. The headline numbers (288 tests, 99.97% line coverage, 4 BUILD commits) are reproducible on a clean clone with `npm ci`.

Track A — Reviewer-only path (no Reddit account needed)

This is the path you take if you want to verify the engineering claims without ever loading the app on a subreddit. ~3 minutes.

A1. Clone, install, type-check, lint

```
git clone https://github.com/vsenthil7/appeal-desk
cd appeal-desk
npm ci
npx tsc --noEmit
npm run lint
```

Pass criteria

- `npm ci` completes with no peer-dep errors. (`@devvit/public-api@0.11.19` resolves.)
- `npx tsc --noEmit` exits 0. No type errors anywhere, including `.tsx` shell.
- `npm run lint` exits 0. (This used to be broken — fix landed in BUILD 003.)

A2. Run the test suite

```
npx vitest run
```

Pass criteria

- All 18 test files pass.
- **288 / 288** tests pass. (Was 197 before the BUILD-003 fixes; +91 tests landed with the pagination, retention, and wiring fixes.)
- Wall-clock under 30 seconds on a modern laptop.

A3. Verify coverage gate

```
npx vitest run --coverage
```

Pass criteria

- Exit code 0 (the gate passes).
- Reported numbers on `core/` + `ai/`:

- statements 99.97%

- branches 99.06%

- functions 100%

- lines 99.97%

- The two non-100% branches are documented in `vitest.config.ts` as defensive `e instanceof Error ? e.message : String(e)` guards. They're practically unreachable; the gate excludes them rather than fake-100ing them.

A4. Run the micro-benchmark

```
npx tsx bench/run.ts
```

Pass criteria — shapes (rough order of magnitude):

Operation	ops/sec
`dedup.computeDedup` (20 prior)	~18,000
`dedup.jaccard`	~110,000
`validation.validateSubmission`	~1,400,000
`validation.sanitiseText`	~1,600,000
`templates.renderTemplate`	~770,000
`rateLimit.checkRateLimit`	~4,000,000
`store.create` (shared author, worst)	~120
`store.openQueuePage` (25)	~4,800+

Don't fail on exact numbers — fail on order of magnitude. The bench is meant to detect a 10× regression, not to win a microbenchmark contest.

Track B — Live install on a test subreddit (~7 minutes)

This is the full product experience. Requires you to be a moderator of a small (<200 members) test subreddit, and to have the Devvit CLI authenticated.

B1. Install the CLI and log in

```
npm install -g devvit
devvit login
```

A browser opens. Sign in with the Reddit account that mods your test sub.

Pass criterion

```
devvit whoami
# Logged in as u/<your username>
```

B2. Install Appeal-Desk on your test sub

Two paths — pick one.

Path 1 — from the App Directory (recommended):

Open https://developers.reddit.com/apps/appeal-desk and click **Install on subreddit**. Pick your test sub.

Path 2 — from source (if you cloned the repo for Track A):

```
cd appeal-desk
devvit upload
devvit install r/<your test sub>
```

Pass criteria

- `devvit list installs` shows `appeal-desk` against your test sub at the latest version.
- Visiting `r/<your test sub>` shows a new menu item under the mod actions (look for "Open Appeals Dashboard").

B3. Trigger a real appeal

1. As a moderator on your test sub, ban a test account you control (or use the second-account trick: a fresh Reddit account that joined the test sub).
2. **Expected**: the banned account receives a civil modmail from your sub inviting them to appeal, with a link to the structured intake form.
3. As the banned account, click the link and fill in the form. The action context (action type, target ID, original removal reason) should be **read-only and pre-filled** — the user can't edit the mod's reason. Write a reason in the "Why this should be reconsidered" paragraph, tick the acknowledgement, submit.

Pass criteria

- The form rejects an empty `reason` with a clear error (validation).
- The form rejects a second submission for the same ban with a clear error (action-lock — duplicate-detection at write time, not just read time).
- Submission succeeds; user sees a confirmation.

B4. Open the dashboard

As a mod, open the new menu item "Open Appeals Dashboard."

Pass criteria

- The submitted appeal appears in the **open** queue.
- Tapping the appeal shows the full detail: the user's submitted reason, the original action snapshot, and (if this user has appealed before) a near-duplicate flag with the Jaccard similarity score.
- Three buttons are visible: **Uphold**, **Overturn**, **Ask for more info**.
- An "AI hint" badge may appear if AI is enabled (it isn't by default).

B5. Make a decision — note the *reply-confirm* gate

Tap **Overturn**. A reply-confirm form opens with a templated civil reply pre-filled.

Pass criterion — **the decision is NOT sent until you press Confirm in this form.** The one-tap button on the previous screen drafts; the reply form is the actual commit point. This is the structural "human decides" guarantee.

Edit the reply if you want. Press Confirm.

B6. Verify the audit trail

Re-open the appeal. The decision now shows on the detail panel with a timestamp, the deciding mod's username, the reply that was sent, and an internal note (if you added one).

Pass criteria

- The decision is permanent — trying to decide again returns `INVALID_STATE_TRANSITION`.
- The appeal has moved out of the **open** queue and into **resolved**.
- The user has received the reply via modmail (check the modmail thread).

B7. Stress the concurrency guarantee (optional, 60 seconds)

In two browser tabs, open the **same** banned user's appeal form for the **same** ban. Submit both at the same time.

Pass criterion

- Exactly one submission succeeds.
- The other gets a clear "this action has already been appealed" error.
- This is enforced by `WATCH/MULTI/EXEC` on `keys.actionLock` — not by hoping no two users press submit in the same millisecond.

B8. (Optional) Verify retention + erasure are wired

Open the app config and set the retention window to 1 day (default is 30).

Resolve an appeal. Wait a day — or, in dev, run the `appealdesk_retention_purge` scheduled job manually via `devvit logs` triggers.

Pass criteria

- The resolved appeal's free-text reason and reply are scrubbed.
- A tombstone row remains in the audit trail (you can see *that* a decision was made; you can no longer see *the text*).
- Calling erasure a second time is a no-op (idempotent).

What "fails fairly"

A few things that look like bugs but aren't:

- **No icon in the dashboard?** The app icon is shipped as `assets/icon.png`, but the Devvit dashboard UI sometimes caches the missing-icon state for ~10 minutes after upload. Refresh after a coffee.
- **Modmail latency.** Reddit modmail isn't instant. Up to ~30 seconds on a busy sub is normal — the decision is *recorded* immediately even if the modmail send takes a moment.
- **AI hint missing.** AI is off by default. Toggle it on in the app settings if you want to see triage labels. The product works fine without it — that is the point.

How to verify the headline claims yourself

Claim	How to verify
288/288 tests pass	`npx vitest run`
99.97% line / 99.06% branch coverage	`npx vitest run --coverage`
4 BUILD commits, all green	`git log --oneline` on `main`
`npm run lint` clean	`npm run lint`
`npx tsc --noEmit` clean	`npx tsc --noEmit`
AI is optional	`grep -n NoopAiProvider src/ai/provider.ts`
Action-lock is CAS, not hopeful	`grep -n claimActionLock src/core/store.ts`
Reply-confirm is structural	`src/components/AppealDetail.tsx` — reply form sits between tap and send
Retention runs daily	`grep -n appealdesk_retention_purge src/server/*.ts`

Thank you

If you got this far, you've done more than most reviewers. The repo welcomes

issues — especially ones that catch errors in this script.

"AI assists. Humans decide. The audit trail proves it."